

Les tubes

75

75

Les tubes

■ Définition d'un tube

- moyen de transmettre un flot d'octets entre des processus
- représenté par une paire de descripteurs associés à une inode présente uniquement en mémoire et gérée de façon spéciale
- unidirectionnel: un côté du tube sert à l'écriture, le second pour la lecture
- la lecture est destructive !
- tous les détails dans **man 7 pipe**
- ordre des octets conservés: FIFO
- capacité finie, qui dépend du système
 - lecteur=processus possédant un descripteur sur le côté lecture
 - écrivain=processus possédant un descripteur sur le côté écriture

76

76

Les tubes

Utilisation des tubes

- Plusieurs processus qui s'exécutent en parallèle peuvent communiquer entre eux
- Il est possible de réaliser cette communication par l'intermédiaire de tubes (pipes)
- Le système est alors chargé d'assurer la synchronisation de l'ensemble des processus créés
- Exemple: `commande1 | commande2 | ... | commanden`



77

Les tubes

Exemple d'un tube

- Un tube est un flux de données permettant l'échange unidirectionnel de données entre deux processus s'exécutant sur une même machine



- Utilisation simple en langage shell (opérateur "|")
- `who | wc -l` » crée deux processus qui s'exécutent en parallèle et qui sont reliés par un tube de communication pipe
- Le premier processus réalise la commande `who`, alors que le second exécute la commande `wc -l`.
- Le processus réalisant la commande `who` ajoute dans le tube une ligne d'information par utilisateur du système.
- Le processus réalisant la commande `wc -l` récupère ces lignes d'information pour en calculer le nombre total

78

Les tubes

❑ Primitives C pour la gestion des tubes :

- **pipe()** : création d'un tube (renvoie une paire de descripteurs de fichier)
- **write()** : écriture dans un tube
- **read()** : lecture du contenu d'un tube
- **close()** : suppression d'un tube
- Un tube de communication sans nom est créé par l'appel système pipe, auquel on passe un tableau de deux entiers : **int pipe(int descripteur[2])**
- Au retour de l'appel système pipe(), un tube aura été créé, et les deux positions du tableau passé en paramètre contiennent deux descripteurs de fichiers
- On peut considérer un descripteur comme une valeur entière que le système d'exploitation utilise pour accéder à un fichier
- Dans le cas du tube, on a besoin de deux descripteurs : le descripteur pour les lectures du tube et le descripteur pour les écritures dans le tube
- Le descripteur de l'accès en lecture se retrouvera à la position 0 du tableau passé en paramètre, et le descripteur de l'accès en écriture se retrouvera à la position 1
- Seul le processus créateur du tube et ses descendants (ses fils) peuvent accéder au tube
- Si le système ne peut pas créer de tube pour manque d'espace, l'appel système pipe() retourne la valeur -1, sinon il retourne la valeur 0

79

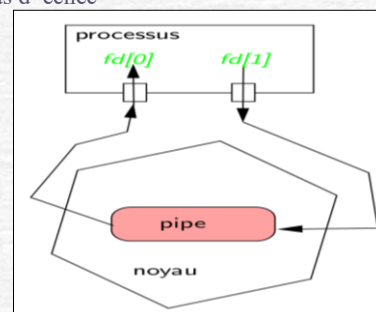
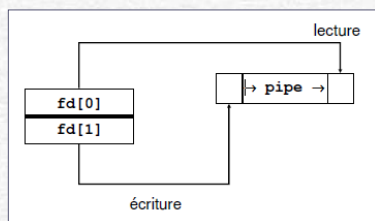
79

Les tubes

❑ Création d'un tube: syntaxe

```
#include <unistd.h>
int pipe(int fildes[2]);
```

- crée deux descripteurs:
 - fildes[0] pour la lecture
 - fildes[1] pour l'écriture
 - retourne 0 en cas de réussite, -1 en cas d'échec



80

80

Les tubes

❑ Lecture : syntaxe

```
#include <sys/types.h>
#include <unistd.h>
ssize_t read(int fd, void *buf, size_t count);
```

- lit **count** octets depuis le descripteur **fd** et les range dans le buffer pointé par **buf**
- renvoie le **nb** d'octets effectivement lus (0 en fin de fichier) et avance la tête de lecture de ce nombre
- renvoie -1 en cas d'échec

81

81

Les tubes

❑ Lecture : fonctionnement

- Si le nb d'écritvains est nul alors la fonction retourne 0
- Si la lecture est bloquante (par défaut) et que le tube est vide, le processus est mis en attente jusqu'à ce qu'une écriture se produise dans le tube
- L'indicateur **O_NONBLOCK** peut être utilisé pour rendre la lecture non bloquante (dans ce cas -1 est renvoyé si le tube est vide)
- Si l'indicateur **O_NDELAY** est positionné et que le tube est vide, alors la fonction retourne 0

❑ Ecriture : syntaxe

```
#include <unistd.h>
ssize_t write(int fd, const void *buf, size_t count);
```

- écrit **count** octets dans le tube désigné par le descripteur **fd** depuis le buffer pointé par **buf**
- renvoie le nombre d'octets effectivement émis
- retourne 0 si aucune écriture n'est réalisée
- renvoie -1 en cas d'échec

82

82

Les tubes

❑ Ecriture: fonctionnement

- Si le nombre de lecteurs est nul, un signal est envoyé au processus écrivain qui se termine
- Si l'écriture est bloquante et que le tube est plein, le processus est bloqué en attendant qu'il se vide
- Si l'écriture est non bloquante, le processus écrit les caractères qu'il peut écrire (maximum **PIPE_BUF**) et retourne une valeur correspondante

❑ Fermeture

- La primitive **close()** doit être utilisée sur le descripteur qu'on désire fermer
 - **close(filedesc[0])** ferme le tube en lecture
 - **close(filedesc[1])** ferme le tube en écriture
- Une fois fermé, on ne peut plus récupérer le descripteur

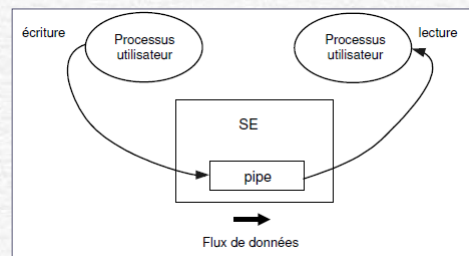
83

83

Les tubes

❑ Fonctionnement de la procédure écriture-lecture

- Le processus père crée un tube de communication sans nom en utilisant l'appel système **pipe()**
- Le processus père crée un ou plusieurs fils en utilisant l'appel système **fork()**
- Le processus écrivain ferme l'accès en lecture du tube
- De même, le processus lecteur ferme l'accès en écriture du tube
- Les processus communiquent en utilisant les appels système **write()** et **read()**
- Chaque processus ferme son accès au tube lorsqu'il veut mettre fin à la communication via le tube



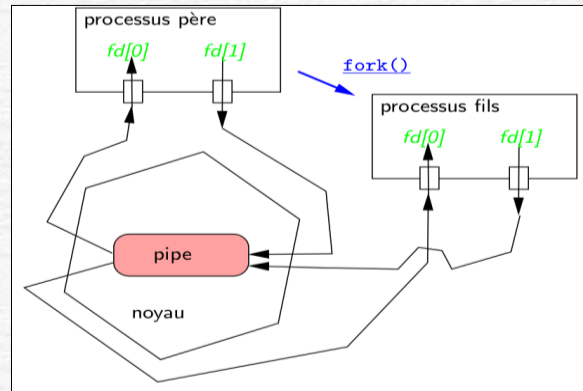
84

84

Les tubes

❑ Pipe – père et fils: fonctionnement

- le fils hérite les descripteurs ouverts dans le père: accès au pipe–communication



85

85

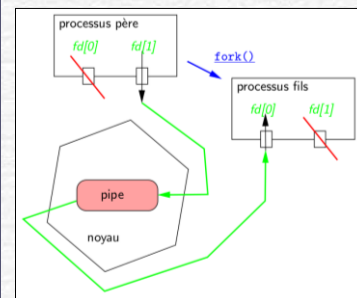
Les tubes

❑ Père parle au fils

- le père ferme fd[0] et écrit sur fd[1]
- le fils ferme fd[1] et lit depuis fd[0]

```
#include<sys/types.h>
#include<unistd.h>
#include <stdio.h>

int main() {
    const int Nbuff=1000; char buff[Nbuff]; int pF ,
    dFic[2];
    pipe(dFic);
    if ((pF = fork()) > 0) { // pere
        close(dFic[0]); write(dFic[1], "Salut", 5);
    }
    else if (pF == 0) { // fils
        close(dFic[1]); int n = read(dFic[0] , buff, Nbuff-1);
        buff[n] = '\0'; printf("buffer=%s\n", buff);
    }
    return 0;
}
```



86

86

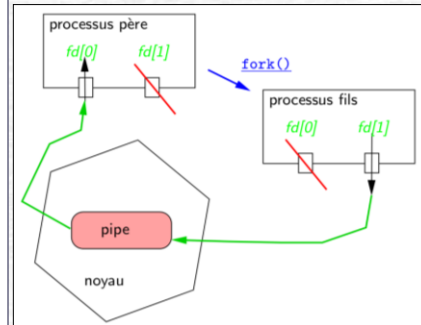
Les tubes

❑ Fils parle au père

- le fils ferme fd[0] et écrit sur fd[1]
- le père ferme fd[1] et lit depuis fd[0]

```
#include<sys/types.h>
#include<unistd.h>
#include <stdio.h>

int main() {
    const int Nbuff=1000; char buff[Nbuff]; int pF ,
    dFic[2];
    pipe(dFic);
    if ((pF = fork()) > 0) { // pere
        close(dFic[1]); int n = read(dFic[0] , buff, Nbuff-1);
        buff[n] = '\0'; printf("buffer=%s\n",buff);
    }
    else if (pF == 0) { // fils
        close(dFic[0]); write(dFic[1], "Salut",5);
    }
    return 0;
}
```



87

87

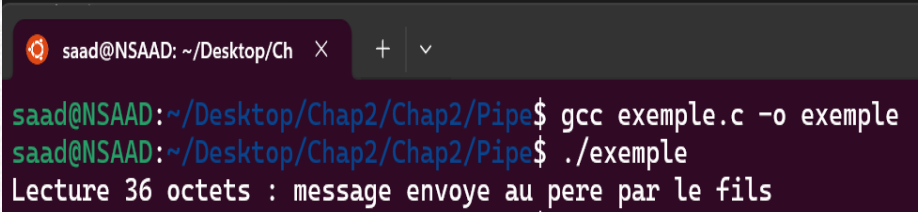
Les tubes

❑ Exemple: le processus père crée un tube de communication pour communiquer

```
#define R 0
#define W 1
int main()
{ int fd[2]; char message [100]; // pour recup. un msg
  int nbocets ; char* phrase = "message envoye au pere par le fils ";
  pipe(fd); // creation d' un tube sans nom
  if (fork()==0) // création d' un processus fils
  { close (fd[R]) ; // Le fils ferme le descripteur non utilise de lecture
    write (fd[W], phrase, strlen(phrase) + 1 ) ; // depot dans le tube du message
    close (fd[W]) ; // fermeture du descripteur d ' ecriture
  }
  else
  { close(fd[W]) ; // Le pere ferme le descripteur non utilise d' écriture
    // extraction du message du tube
    nbocets = read(fd[R], message, 100);
    printf ( "Lecture %d octets : %s\n", nbocets, message);
    close (fd[R]) ; // fermeture du descripteur de lecture
  }
  return 0 ;
}
```

88

88



```
saad@NSAAD: ~/Desktop/Ch X + v
saad@NSAAD:~/Desktop/Chap2/Chap2/Pipe$ gcc exemple.c -o exemple
saad@NSAAD:~/Desktop/Chap2/Chap2/Pipe$ ./exemple
Lecture 36 octets : message envoye au pere par le fils
```

89

89

Les tubes

❑ Exercice

- On veut créer deux processus qui échangent des entiers de la manière suivante
- Le père crée un tube
- Le père crée un fils
- Le fils lit une suite d'entiers au clavier terminée par Ctrl-D
- Il en fait la somme
- Il l'écrit dans le tube au père
- Le père lit la somme dans le tube puis l'affiche

90

90

Les tubes

❑ Solution

```
int main () {
    pid_t pid; int n, somme=0, p[2]; pipe(p);
    /* processus fils */
    if (fork()==0) {
        close(p[0]); printf("Entiers (fin E+D)\n");
        while (scanf ("%d",&n)!=EOF)
            somme+=n;
        write(p[1],&somme,sizeof(int)); close(p[1]);
        exit(0);
    }
    /* processus pere */
    else {
        close(p[1]); read(p[0],&somme,sizeof(int));
        printf("somme=%d\n",somme); close(p[0]);
        wait(&n);
        exit(0);
    }
}
```

- ❑ On souhaite maintenant que le fils envoie tous les entiers lus au père, c'est le père qui est chargé de faire la somme avant de l'afficher

91

91

Les tubes

- ❑ On souhaite maintenant que le fils envoie tous les entiers lus au père, c'est le père qui est chargé de faire la somme avant de l'afficher

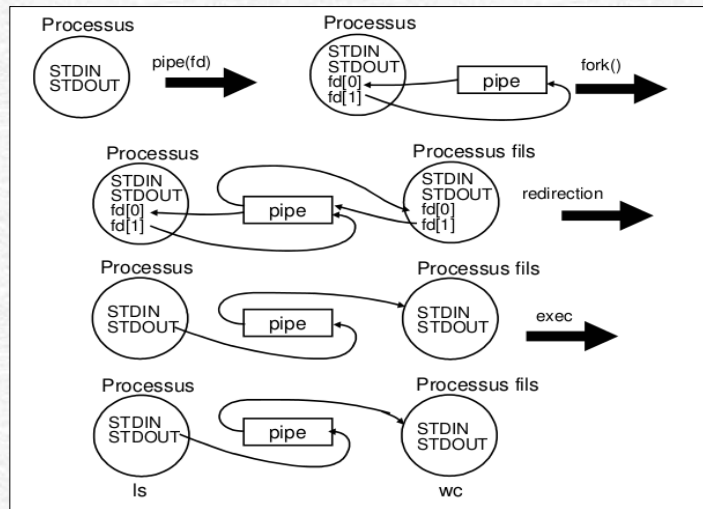
```
int main () {
    pid_t pid; int n,somme=0,p[2]; pipe(p);
    if (fork()==0) { /* processus fils */
        close(p[0]); printf("Entiers (fin E+D)\n");
        while (scanf ("%d",&n)!=EOF)
            write(p[1],&n,sizeof(int));
        close(p[1]);
        exit(0);
    }
    else { /* processus pere */
        close(p[1]);
        while (read(p[0],&n,sizeof(int)))
            somme+=n;
        read(p[0],&somme,sizeof(int)); printf("somme=%d\n",somme); close(p[0]);
        wait(&n);
        exit(0);
    }
}
```

92

92

Les tubes

□ Expliquez la figure suivante



93

93

Les tubes

□ Expliquez la figure suivante

- Le tube est créé avec la commande `pipe()`
- Puis un processus fils est créé. Ce processus fils héritera des accès en entrée et en sortie du tube.
- Le processus parent ferme son accès à la sortie du tube et fait pointer **STDOUT** sur l'entrée du tube.
- Le processus fils ferme son accès à l'entrée du tube et redirige **STDIN** sur la sortie du tube.
- Les deux processus sont remplacés par les programmes `ls` et `wc`.

94

94

Les tubes

```
#include <stdlib.h>
#include <stdio.h>
void runpipe();
int main(int argc, char **argv)
{
    int pid, status;
    int fd[2];
    pipe(fd);
    switch (pid = fork()) {
        case 0: /* child */
            runpipe(fd);
            exit(0);
        default: /* parent */
            while ((pid = wait(&status)) != -1)
                fprintf(stderr, "process %d exits with %d\n", pid, WEXITSTATUS(status)); break;
        case -1: perror("fork"); exit(1);
    }
    exit(0);
}

char *cmd1[] = { "ls", "-l", "/", 0 };
char *cmd2[] = { "tr", "a-z", "A-Z", 0 };

void runpipe(int pfd[])
{
    int pid;
    switch (pid = fork())
    {
        case 0: /* child */
            dup2(pfd[0], 0);
            close(pfd[1]); /* the child does not need this end of the pipe */
            execvp(cmd2[0], cmd2);
            perror(cmd2[0]);
        default: /* parent */
            dup2(pfd[1], 1);
            close(pfd[0]); /* the parent does not need this end of the pipe */
            execvp(cmd1[0], cmd1); perror(cmd1[0]);
        case -1:
            perror("fork"); exit(1);
    }
}
```

95

```
saad@NSAAD: ~/Desktop/Chap2/Chap2/Pipe$ gcc process.c -o process
saad@NSAAD: ~/Desktop/Chap2/Chap2/Pipe$ ./process
TOTAL 2796
LRwxRwxRwx 1 ROOT ROOT 7 APR 22 2024 BIN -> USR/BIN
DRwxR-XR-X 2 ROOT ROOT 4096 FEB 26 2024 BIN.USR-IS-MERGED
DRwxR-XR-X 2 ROOT ROOT 4096 APR 22 2024 BOOT
DRwxR-XR-X 15 ROOT ROOT 3860 FEB 22 11:04 DEV
DRwxR-XR-X 89 ROOT ROOT 4096 FEB 23 09:21 ETC
DRwxR-XR-X 3 ROOT ROOT 4096 FEB 17 19:35 HOME
-rwxR-XR-X 1 ROOT ROOT 2781568 DEC 12 02:58 INIT
LRwxRwxRwx 1 ROOT ROOT 7 APR 22 2024 LIB -> USR/LIB
DRwxR-XR-X 2 ROOT ROOT 4096 APR 8 2024 LIB.USR-IS-MERGED
LRwxRwxRwx 1 ROOT ROOT 9 APR 22 2024 LIB64 -> USR/LIB64
DRwx----- 2 ROOT ROOT 16384 FEB 17 19:34 LOST+FOUND
DRwxR-XR-X 2 ROOT ROOT 4096 JAN 6 2025 MEDIA
DRwxR-XR-X 5 ROOT ROOT 4096 FEB 17 19:34 MNT
DRwxR-XR-X 2 ROOT ROOT 4096 JAN 6 2025 OPT
DR-XR-XR-X 286 ROOT ROOT 0 FEB 22 11:03 PROC
DRwx----- 4 ROOT ROOT 4096 FEB 17 19:34 ROOT
DRwxR-XR-X 20 ROOT ROOT 600 FEB 23 09:20 RUN
LRwxRwxRwx 1 ROOT ROOT 8 APR 22 2024 SBIN -> USR/SBIN
DRwxR-XR-X 2 ROOT ROOT 4096 MAR 31 2024 SBIN.USR-IS-MERGED
DRwxR-XR-X 2 ROOT ROOT 4096 FEB 17 19:34 SNAP
DRwxR-XR-X 2 ROOT ROOT 4096 JAN 6 2025 SRV
DR-XR-XR-X 13 ROOT ROOT 0 FEB 22 11:03 SYS
DRwxRwxRwT 12 ROOT ROOT 4096 FEB 23 10:36 TMP
DRwxR-XR-X 12 ROOT ROOT 4096 JAN 6 2025 USR
DRwxR-XR-X 13 ROOT ROOT 4096 FEB 17 19:34 VAR
process 50311 exits with 0
```

96

96

Les tubes

- ❑ Comment créer un pipe entre deux processus fils: `ls -al | tr a-z A-Z`. Donner le code des fonctions `runsource()` et `rundest()`

```
#include <stdlib.h>
#include <stdio.h>

char *cmd1[] = { "/bin/ls", "-al", "/", 0 };
char *cmd2[] = { "/usr/bin/tr", "a-z", "A-Z", 0 };

void runsource(int pfd[]);
void rundest(int pfd[]);

int main(int argc, char **argv)
{ int pid, status;
  int fd[2]; pipe(fd);
  runsource(fd);
  rundest(fd);
  close(fd[0]);
  close(fd[1]); /* this is important! close both file descriptors on the pipe */
  while ((pid = wait(&status)) != -1) /* pick up all the dead children */
    fprintf(stderr, "process %d exits with %d\n", pid, WEXITSTATUS(status));
  exit(0);
}
```

97

97

Les tubes

- ❑ Comment créer un pipe entre deux processus fils: `ls -al | tr a-z A-Z`. Donner le code des fonctions `runsource()` et `rundest()`

```
void runsource(int pfd[]) /* run the first part of the pipeline, cmd1 */
{ int pid; /* we don't use the process ID here, but you may want to print it for debugging */
  switch (pid = fork())
  { case 0: /* child */
    dup2(pfd[1], 1); /* this end of the pipe becomes the standard output */
    close(pfd[0]); /* this process don't need the other end */
    execvp(cmd1[0], cmd1); /* run the command */
    perror(cmd1[0]); /* it failed! */
    default: /* parent does nothing */ break;
    case -1: perror("fork"); exit(1);
  }
}

void rundest(int pfd[]) /* run the second part of the pipeline, cmd2 */
{ int pid;
  switch (pid = fork())
  { case 0: /* child */
    dup2(pfd[0], 0); /* this end of the pipe becomes the standard input */
    close(pfd[1]); /* this process doesn't need the other end */
    execvp(cmd2[0], cmd2); /* run the command */
    perror(cmd2[0]); /* it failed! */
    default: /* parent does nothing */ break;
    case -1: perror("fork"); exit(1);
  }
}
```

98

98

Les tubes

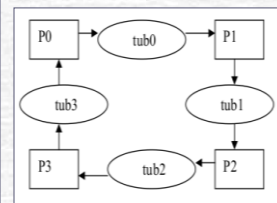
❑ Comment créer un ring de processus:

- Complétez ce code de manière à implémenter cette architecture de communication des N processus créés.
- L'entrée standard et la sortie standard de chaque processus P_i sont redirigées vers les tubes appropriés.
- Par exemple, pour le processus P_0 , l'entrée et la sortie standards deviennent respectivement les tubes $tub3$ et $tub0$.

```
#define N 4

void proc( int );

int main ( )
{
    int i ;
    for( i=0 ; i <N; i++)
        if ( fork() ==0)
            {      proc(i); exit(0);      }
    exit(0);
}
```



99

99

Les tubes

❑ Comment créer un ring de processus

```
#include<stdlib.h>
#include<stdio.h>
#include<sys/types.h>
#include<sys/types.h>
#include<unistd.h>

#define N 4
void proc(int i) ; // Declaration en avant

/* On cree les pipes, puis le N processus fils */
int main()
{
    int i ;
    int j ;
    int fd[N][2]; /* Descripteurs de tous les pipes */
    for(i=0; i<N; i++) {pipe(fd[i]);} // creation de tous les pipes
    /*Le processus i lit sur le pipe i et ecrit sur le pipe i+1, les calculs d'indice se font en utilisant modulo N*/
    for(i=0; i<N; i++)
    {
        if (fork() ==0)
        { // on redirige entree et sortie standard
            dup2(fd[i][1],1);
            dup2(fd[(N+i-1)%N][0],0);
            // Fermer tous les descripteurs de pipes
            for(j=0; j<N; j++) {close(fd[j][0]); close(fd[j][1]);}
            // proc(i);
            exit(0);
        }
    }
    exit(0);
}
```

100

100

Les tubes

- ❑ Modifier le programme du ring pour mettre en ouvre la commande suivante:
ls -al / | tr a-z A-Z

101

101

Les tubes

```
int main (int argc, char *argv[]){
    int Pid;
    int Fils,Etat;
    if (argc != 2){
        printf(" Utilisation : %s fic. a executer ! \n", argv[0]);
        exit(1);
    }

    printf (" Je suis le processus %d je vais faire fork\n",(int)getpid());
    Pid=fork();
    switch (Pid){
        case 0 :
            printf (" Je suis le fils %d\n", (int)getpid());
            printf (" %d : Code remplace par %s\n", (int)getpid(), argv[1]);
            execl(argv[1], argv[1], NULL);
            printf (" %d : Erreur lors du exec \n", (int)getpid());
            exit (2);
        case -1 :
            printf ("Le fork n'a pas reussi ");
            exit (3);
        default :
            printf (" Pere numero %d attend\n ",(int) getpid());
            Fils=wait(&Etat);
            printf ( " Le fils etait : %d \n", Fils);
            exit(0);
    }
    return 0;
}
```

102

102

Les tubes

❑ Modifier le programme du ring pour mettre en oeuvre la commande suivante:
ls -al / | tr a-z A-Z

❑ Donner un programme pour mettre en oeuvre les commandes suivantes:

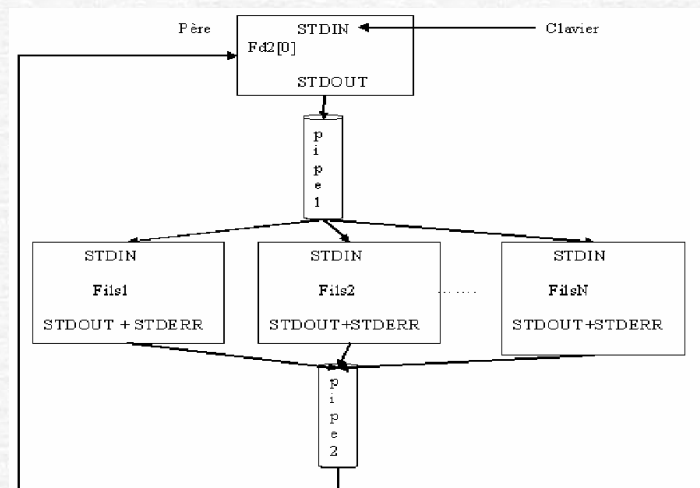
- ps | grep emacs | wc -l
- sort telephone.txt | grep Dupond | uniq -u | cut -f 3

Durand	Emilie	0381818585
Terieur	Alex	0478858689
Tinrieur	Georges	0563868985
Dupond	Albert	04961868957
Dupont	Emilie	02971457895
Dupond	Albertine	0131986258
Bouvier	Jacques	0381698759
Zeblues	Agathe	0685987456
Dupond	Agnès	0687598614
Dumont	Patrick	04661645987
Dupond	Elisabeth	0654896325
Houtand	Laurent	0658769458

103

103

Les tubes



104

104

Les tubes

```
#define N 2

void gestion(){ /* traitement */ };
/*0*/ int fd1[2], fd2[2];

int main( )
{
    pid_t pid[N];
    int i;

    /*1*/ pipe(fd1); pipe(fd2);

    for ( i=0; i<N;i++)
        if ((pid[i]=fork())==0)
        { /* 2*/ dup2(fd1[0],0) ; dup2(fd2[1],1) ; dup2(fd2[1],2) ;
            close(fd1[0]); close(fd1[1]); close(fd2[1]); close(fd2[0]);
            //execlp("traitement", "traitement", NULL);
            execlp("ls", "-al", "/", NULL);

            exit(1);
        }
    /*3*/ dup2(fd1[1],1); close(fd1[0]); close(fd1[1]); close(fd2[1]);
    // gestion( );

    return 0;
}
```

105

105

Les signaux

106

106

Les signaux

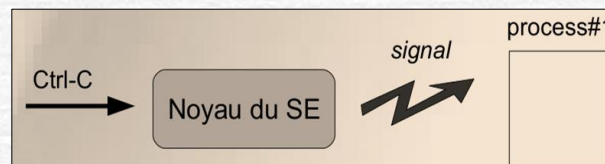
- ❑ Un signal est l'équivalent logiciel des interruptions matérielles
- ❑ Exemple : un utilisateur demande à un éditeur de charger un très gros fichier, il peut réaliser son erreur et demander d'interrompre le chargement
- ❑ Technique habituelle : frappe de touches particulières (Ctrl-C, DEL ...) qui envoie un signal au processus éditeur ; ce dernier récupère le signal et interrompt le chargement
- ❑ La prise en compte du signal déclenche l'exécution d'une fonction particulière définie par le système ou l'utilisateur
- ❑ Les signaux sont identifiés par des nombres entiers : 1 ... NSIG
- ❑ Des constantes sont définies dans **signal.h**

107

107

Les signaux

- ❑ A chaque signal est associé un comportement par défaut :
 - **exit** : terminaison du processus
 - **ignore** : le signal est ignoré
 - **stop** : suspension du processus
 - **continue** : reprendre l'exécution si le processus a été suspendu



108

108

Les signaux

Numéro	Nom	Comportement par défaut
1	SIGUP	envoyé lors de la déconnexion
2	SIGINT	interruption clavier
3	SIGQUIT	quitter depuis le clavier
6	SIGABRT	terminaison anormale
8	SIGFPE	erreur opération en virgule flottante
9	SIGKILL	destruction inconditionnelle du processus
10	SIGUSR1	à disposition de l'utilisateur
11	SIGSEGV	référence mémoire invalide
12	SIGUSR2	à disposition de l'utilisateur
13	SIGPIPE	erreur dans un tube (écriture sans lecteur)
14	SIGALARM	signal d'horloge
15	SIGTERM	terminaison normale d'un processus
18	SIGCONT	reprise d'exécution, si stoppé
19	SIGSTOP	stoppe l'exécution d'un processus

109

109

Les signaux

□ On peut obtenir la liste des signaux par la commande **kill -l**

```
saad@NSAAD:~/Desktop/Chap2/Chap2/Pipe$ kill -l
1) SIGHUP      2) SIGINT      3) SIGQUIT     4) SIGILL      5) SIGTRAP
6) SIGABRT     7) SIGBUS     8) SIGFPE     9) SIGKILL    10) SIGUSR1
11) SIGSEGV    12) SIGUSR2    13) SIGPIPE    14) SIGALRM    15) SIGTERM
16) SIGSTKFLT  17) SIGCHLD   18) SIGCONT    19) SIGSTOP    20) SIGTSTP
21) SIGTTIN    22) SIGTTOU   23) SIGURG     24) SIGXCPU   25) SIGXFSZ
26) SIGVTALRM  27) SIGPROF   28) SIGWINCH   29) SIGIO      30) SIGPWR
31) SIGSYS     34) SIGRTMIN  35) SIGRTMIN+1 36) SIGRTMIN+2 37) SIGRTMIN+3
38) SIGRTMIN+4 39) SIGRTMIN+5 40) SIGRTMIN+6 41) SIGRTMIN+7 42) SIGRTMIN+8
43) SIGRTMIN+9 44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9 56) SIGRTMAX-8 57) SIGRTMAX-7
58) SIGRTMAX-6 59) SIGRTMAX-5 60) SIGRTMAX-4 61) SIGRTMAX-3 62) SIGRTMAX-2
63) SIGRTMAX-1 64) SIGRTMAX
```

110

110

Les signaux

❑ Rappel:

- 3 façons d'envoyer un signal à un processus :
 - En ligne de commande :

```
kill [-no_signal] no_processus
kill -9 4014

kill [-nom_signal] no_processus
kill -SIGUSR1 2584
```

- Un caractère de contrôle :

CRTL-C	envoi SIGINT (interruption)
CRTL-Z	envoi SIGSTOP (suspension)
CRTL-\	envoi SIGQUIT (fin)

- Primitives C de gestion des signaux :
 1. `kill()` : envoi un signal vers un processus
 2. `sigaction()` : spécifie une action associée à un signal

111

111

Les signaux

❑ Syntaxe: `kill -SIGNAL PID`

- envoi le signal `SIGNAL` au processus de pid `PID`
- `kill -KILL 7657` ou `kill -9 7657` envoi le signal `KILL` (tue) au processus de pid 7657
- `kill -STOP 7765` envoi le signal `STOP` (suspend) au processus de pid 7765
- `kill -CONT 7765` envoi le signal `CONT` (reprend l'exécution) au processus de pid 7765

❑ Syntaxe d'envoi d'un signal `sig` au processus `pid`

```
#include<signal.h>
int kill(pid_t pid, int sig);
```

- `pid` peut prendre les valeurs suivantes :
 - `> 0` le signal est envoyé au processus de pid `pid`
 - `= 0` le signal est envoyé à tous les processus du même groupe que le processus émetteur
 - `< -1` tous les processus du même groupe que le processus de pid `pid`
- `sig` peut prendre les valeurs suivantes:
 - `= 0` teste l'existence du processus
 - `1...NSIG` envoi le signal spécifié

112

112

Les signaux

Exemple 1

```
#include <unistd.h>
#include <stdio.h>
#include <signal.h>
// #include <stdlib.h>
#include <sys/wait.h>
int main() {
    int pid, statut;
    if ((pid=fork())==0)
        while(1) sleep(1);
    else {
        sleep(10); statut = kill(pid,0);
        if (statut == -1) printf("Fils %d inexistant \n",pid);
        else {
            printf("Signal SIGKILL envoye a %d\n",pid);
            kill(pid,SIGKILL);
            pid = waitpid(pid,&statut,WUNTRACED);
            printf("Le fils %d s'est termine\n",pid);
        }
    }
}
```

```
saad@NSAAD:~/Desktop/Chap2/Chap2/Signal$ gcc Ex1.c -o Ex1
saad@NSAAD:~/Desktop/Chap2/Chap2/Signal$ ./Ex1
Signal SIGKILL envoye a 53218
Le fils 53218 s'est termine
```

113

113

Les signaux

Prise en compte des signaux: principe

- A chaque signal est associé un comportement (handler) par défaut
- Le comportement de certains signaux (SIGKILL, SIGCONT et SIGSTOP) ne peut pas être modifié
- Pour modifier le comportement des autres signaux, il faut leur associer un nouvel handler
- Lors de la prise en compte du signal, la fonction handler est exécutée ; le processus reprend ensuite au retour de la fonction

Définition d'un comportement : syntaxe

```
int sigaction(int signum, const struct sigaction *newact,
              struct sigaction *oldact);
```

- Cet appel système sert à modifier l'action effectuée à la réception d'un signal
- La fonction retourne 0 normalement, -1 en cas de problème
- *signum* désigne le signal concerné
- *newact* est le nouveau comportement
- *oldact* permet de sauvegarder l'ancien comportement

114

114

Les signaux

■ La structure de comportement **sigaction**

```
#include<signal.h>
struct sigaction {
    union {
        void (*__sa_handler) (int);
        void (*__sa_sigaction) (int,struct __siginfo *,void *);
    } __sigaction_u; /* signal handler */
    int      sa_flags; /* see signal options below */
    sigset_t sa_mask; /* signal mask to apply */
};
#define sa_handler      __sigaction_u.__sa_handler
#define sa_sigaction    __sigaction_u.__sa_sigaction
```

- **sa_handler** indique l'action à effectuer (SIG DFL par défaut, SIG IGN pour ignorer ou un pointeur vers une fonction définissant le comportement)
- **sa_mask** fournit un masque de signaux à bloquer
- **sa_flags** spécifie un ensemble d'attributs qui modifie le comportement du gestionnaire de signaux

115

115

Les signaux

■ Exemple 2

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
int cpt=0;
void comportement(int sig) {
    if (sig == SIGUSR1) cpt++;
    else if (sig == SIGINT) {
        printf ("nombre signaux recus : %d\n",cpt+1);
        exit(0);
    }
}

int main() {
    struct sigaction action;
    action.sa_handler=comportement;
    sigaction(SIGUSR1,&action,NULL);
    sigaction(SIGINT,&action,NULL);
    while (1);
}
```

116

116

Les signaux

Exemple 2

- Terminal 1

```
saad@NSAAD:~/Desktop/Chap2/Chap2/Signal$ ./Ex2
^Cnombre signaux recus : 6
saad@NSAAD:~/Desktop/Chap2/Chap2/Signal$
```

- Terminal 2

```
saad@NSAAD:~/Desktop/Chap2/Chap2/Signal$ ps aux | grep Ex2
saad  53776  108  0.0  2548  1024 pts/7    R+   10:57   0:11 ./Ex2
saad  53808  0.0  0.0  4088  1920 pts/6    S+   10:57   0:00 grep --color=auto Ex2
saad@NSAAD:~/Desktop/Chap2/Chap2/Signal$ kill -USR1 53776
saad@NSAAD:~/Desktop/Chap2/Chap2/Signal$ kill -USR1 53776
saad@NSAAD:~/Desktop/Chap2/Chap2/Signal$ kill -USR1 53776
saad@NSAAD:~/Desktop/Chap2/Chap2/Signal$ kill -USR1 53776
saad@NSAAD:~/Desktop/Chap2/Chap2/Signal$ kill -USR1 53776
saad@NSAAD:~/Desktop/Chap2/Chap2/Signal$
```

117

117

Les signaux

Exemple 3

```
#include<stdio.h>
#include<signal.h>
#include<unistd.h>

void sighandler(int signum)
{ printf("Signal SIGTERM intercepte \n"); }

int main ( void )
{ char buffer [256] ;
  if(signal(SIGTERM,&sighandler) == SIG_ERR)
  { printf("Ne peut pas manipuler le signal \n" ) ; return 1 ; }
  while ( 1 )
  { fgets(buffer, sizeof(buffer), stdin) ; printf ("Input : %s ", buffer) ; }
  return 0 ;
}
```

- Terminal 1: ./Ex3
Salut
Input: salut
- Terminal 2: kill -SIGKILL npid

118

118

Références

- Introduction aux systèmes et aux réseaux (S. Krakowiak, Grenoble), Source de nombreux transparents présentés ici.
<http://sardes.inrialpes.fr/~krakowia/Enseignement/L3/SR-L3.html>
- Cours du Mr J. Chroboczek, 2013
- Cours de Y. PAGNOTTE, Systèmes d'exploitation et programmation système
- Luigi Logrippo, Ordonnancement Processus, slides. <http://w3.uqo.ca/luigi/>
- <http://perso.univ-rennes1.fr/pascal.gentil/docs/unix/unix.fichiers.html#cat1>
- <http://upsilon.cc/~zack/teaching/1314/progsyst/#index1h2>
- Karim Sehaba, Systèmes et Architectures, <http://perso.univ-lr.fr/ksehaba/>
- Vincent Granet, Introduction à Linux, Polytech'Nice-Sophia
- Sylvain CHERRIER, Cours Linux,
- C. F Zucke, cours Systèmes d'exploitation
- Gestion des processus, Telecom-ParisTech BCI Informatique